EECE 503Q - DevSecOps

Project ShopCloud

E-Commerce Platform

Phase 1: Architecture and Design Report

American University of Beirut

Maroun Semaan Faculty of Engineering & Architecture

Spring 2026

Table of Contents

1. Executive Summary **3**

2. Architecture Overview **4**

2.1 How Traffic Flows Through the System 7

2.2 ECS Fargate - Compute Layer 7

2.3 Database, Async Pipeline, and Disaster Recovery 7

2.4 Network Security Model 7

2.5 VPC and Subnet Layout 8

3. Service-to-Infrastructure Mapping **9**

3.1 Product Catalog 9

3.2 Shopping Cart 10

3.3 Checkout & Payment 11

3.4 Authentication 12

3.5 Inventory Admin Panel 13

3.6 Orders & Inventory Database 14

3.7 Invoice Generation 15

3.8 Dev & Production Environments 16

4. Cost Estimation **17**

4.1 Production Environment 17

4.2 Development Environment 18

4.3 Combined Summary and Cost Drivers 18

5. Design Methodology - Key Decisions **19**

5.1 Compute: ECS Fargate + Lambda 19

5.2 Environment Isolation: Separate VPCs 20

5.3 Region Strategy: Single Region + CDN + DR 20

5.4 Database: RDS PostgreSQL Multi-AZ 21

5.5 Invoice Pipeline: SQS + Lambda + S3 + SES 21

5.6 Admin Access: Internal ALB + Bastion 22

6. Conclusion **23**

1. Executive Summary

ShopCloud is a lightweight e-commerce platform designed for a startup targeting customers in Europe and the United States. This report presents the AWS architecture for eight services: product catalog, shopping cart, checkout, authentication, admin panel, relational database, invoice generation, and isolated dev/prod environments. Every decision is justified on technical and economic grounds, with alternatives explicitly evaluated.

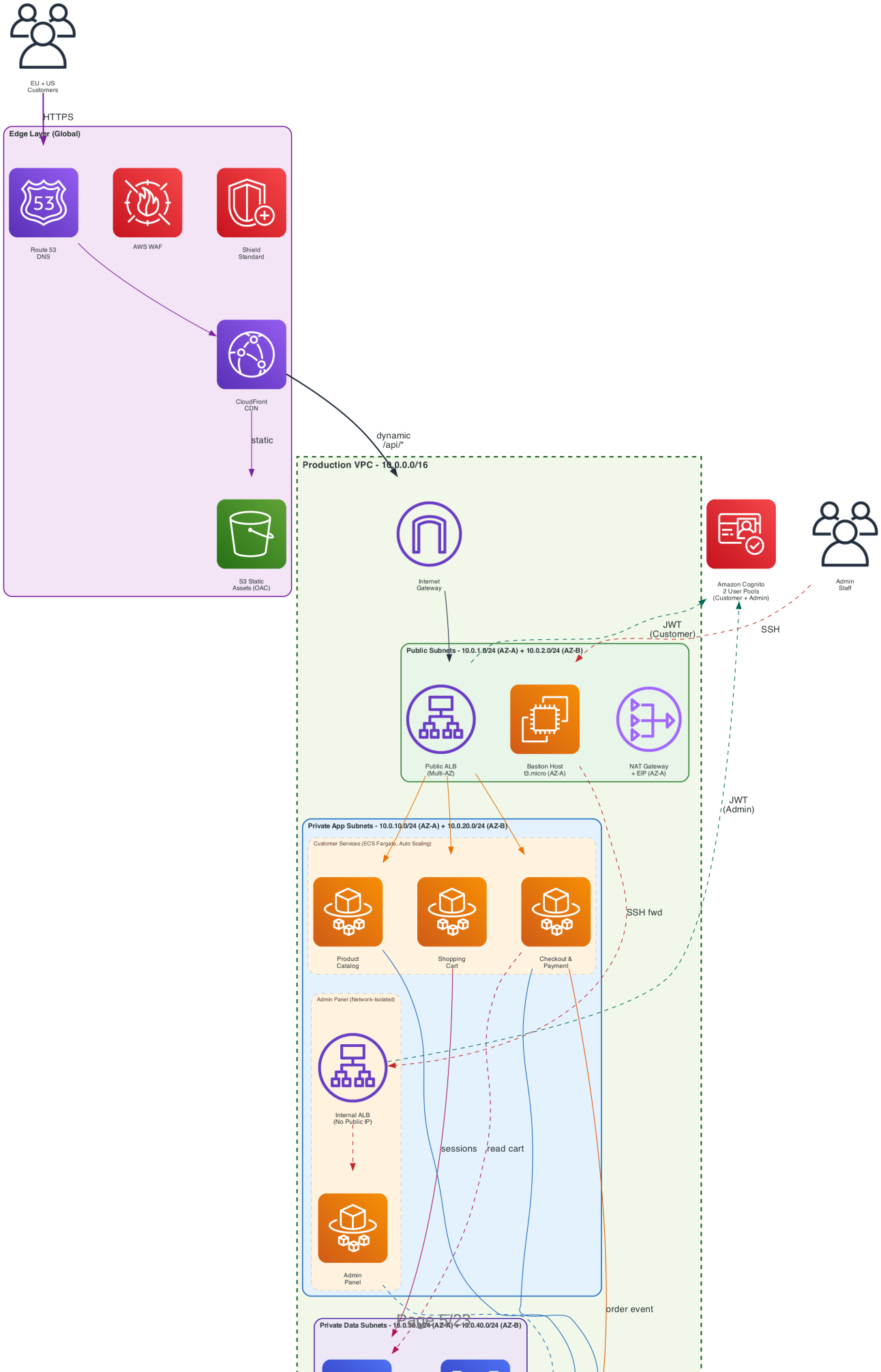
Key architectural choices: ECS Fargate runs four synchronous services as independent, auto-scaling containers with no cluster management overhead. RDS PostgreSQL Multi-AZ provides automatic failover with a cross-region replica in eu-west-1 (Ireland) for European customers. Amazon Cognito handles authentication with two user pools (customer + admin) at zero cost. The admin panel is triply isolated behind an Internal ALB, bastion SSH tunnel, and Cognito Admin Pool. An async invoice pipeline (SQS + Lambda + S3 + SES + DLQ) operates entirely within free tier. Production and development run in separate VPCs (10.0.0.0/16 and 10.1.0.0/16) with three-tier subnets and no peering. Edge security uses Route 53, CloudFront, WAF, and Shield Standard.

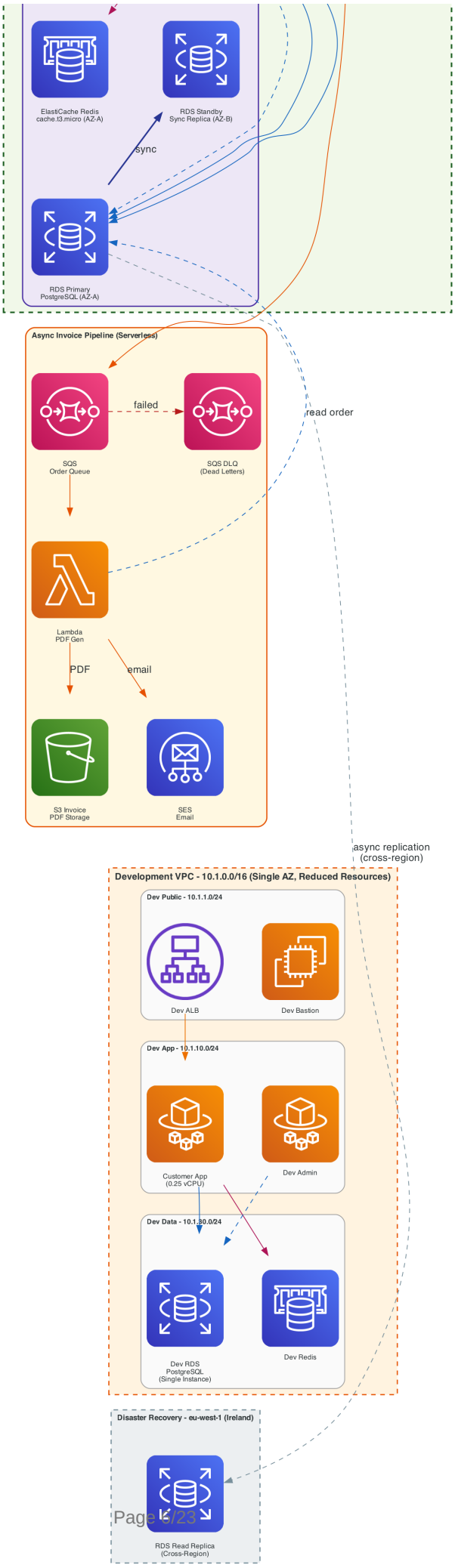
The estimated monthly cost is $225.49 for production and $117.04 for development, totaling $342.53/month ($4,110 annually) - 25% less than an equivalent EKS architecture by eliminating $237/month in Kubernetes control plane and worker node fees. The entire infrastructure is designed to be provisioned via Terraform from a single AWS account.

2. Architecture Overview

The following diagram shows the complete ShopCloud architecture, deployed in the us-east-1 (N. Virginia) AWS region. The production VPC spans two Availability Zones (AZ-A and AZ-B) with three-tier subnet isolation (public, private app, private data). A separate development VPC mirrors the production architecture at reduced scale. The edge layer, async invoice pipeline, and disaster recovery region are shown alongside both VPCs.

Figure 1: ShopCloud Complete Architecture (Part 1 of 2 - Edge, VPC, Services)


EU + US Customers
HTTPS
Edge Layer (Global)
Route 53 DNS
AWS WAF
Shield Standard
CloudFront CDN
static
S3 Static Assets (OAC)
dynamic /api/*
Production VPC - 10.0.0.0/16
Internet Gateway
Amazon Cognito 2 User Pools (Customer + Admin)
Admin Staff
JWT (Customer)
SSH
Public Subnets - 10.0.1.0/24 (AZ-A) + 10.0.2.0/24 (AZ-B)
Public ALB (Multi-AZ)
Bastion Host t3.micro (AZ-A)
NAT Gateway + EIP (AZ-A)
JWT (Admin)
Private App Subnets - 10.0.10.0/24 (AZ-A) + 10.0.20.0/24 (AZ-B)
Customer Services (ECS Fargate, Auto Scaling)
Product Catalog
Shopping Cart
Checkout & Payment
SSH fwd
Admin Panel (Network-Isolated)
Internal ALB (No Public IP)
sessions
read cart
Admin Panel
order event
Private Data Subnets - 10.0.50.0/24 (AZ-A) + 10.0.40.0/24 (AZ-B)


ElastiCache Redis
cache.t3.micro (AZ-A)
RDS Standby
Sync Replica (AZ-B)
sync
RDS Primary
PostgreSQL (AZ-A)
Async Invoice Pipeline (Serverless)
failed
read order
SQS
Order Queue
SQS DLQ
(Dead Letters)
Lambda
PDF Gen
PDF
email
S3 Invoice
PDF Storage
SES
Email
async replication
(cross-region)
Development VPC - 10.1.0.0/16 (Single AZ, Reduced Resources)
Dev Public - 10.1.1.0/24
Dev ALB
Dev Bastion
Dev App - 10.1.10.0/24
Customer App
(0.25 vCPU)
Dev Admin
Dev Data - 10.1.30.0/24
Dev RDS
PostgreSQL
(Single Instance)
Dev Redis
Disaster Recovery - eu-west-1 (Ireland)
RDS Read Replica
(Cross-Region)

2.1 How Traffic Flows Through the System

Static assets (images, CSS, JS) are served through CloudFront, which caches at edge locations near both EU and US customers (~10ms vs ~100ms cross-Atlantic). The S3 origin bucket is not publicly accessible - CloudFront accesses it via Origin Access Control (OAC).

Dynamic API requests enter the VPC through the Internet Gateway and reach the Public ALB in the public subnets. The ALB performs path-based routing (/api/products/*, /api/cart/*, /api/checkout/*) to the appropriate Fargate service and integrates with Cognito for JWT validation. The ALB spans both AZs for high availability. The Bastion Host (t3.micro, AZ-A) provides SSH access for admin panel access. The NAT Gateway enables outbound calls from private subnets (ECR image pulls, AWS API calls) without inbound exposure.

2.2 ECS Fargate - Compute Layer

ECS Fargate runs all application containers with no cluster management. Each service has its own auto scaling policy, task definition, and security group. Three customer-facing services (Catalog, Cart, Checkout) are registered on the Public ALB. The Admin Panel runs behind an Internal ALB with no public IP - accessible only via bastion SSH tunnel, protected by the Cognito Admin Pool. ElastiCache Redis (cache.t3.micro) in the data subnet serves as the shared session store for Cart and Checkout, persisting sessions across task restarts.

2.3 Database, Async Pipeline, and Disaster Recovery

RDS PostgreSQL Multi-AZ (db.t3.micro) stores all persistent data. The primary runs in AZ-A with a synchronous standby in AZ-B (RPO = 0, failover in 60-120 seconds). Automated snapshots run daily with 35-day retention. Three Fargate services and the Lambda invoice function connect to RDS; the Shopping Cart uses only ElastiCache Redis.

The async invoice pipeline decouples PDF generation from checkout: Checkout publishes to SQS, Lambda generates the PDF, uploads it to S3 (for re-download), and emails a link via SES. Failed messages move to a DLQ after max retries - no invoice is silently lost. The entire pipeline operates within AWS free tier.

A cross-region read replica in eu-west-1 (Ireland) provides geographic redundancy close to European customers. It serves both as a DR target and a low-latency read endpoint for EU traffic. Cost: ~$15/month.

2.4 Network Security Model

Defense-in-depth with eight security groups using SG-to-SG references (identity-based, not CIDR-based). The edge layer (WAF, Shield Standard) filters malicious traffic before it enters the VPC. Cognito validates JWT tokens at the ALB. Key SG chain:

Security Group	Inbound From	Protects
Public ALB SG	CloudFront only (443)	VPC entry point

Customer Fargate SG	Public ALB SG	Catalog, Cart, Checkout
Internal ALB SG	Bastion SG only	Admin entry point (no public IP)
Admin Fargate SG	Internal ALB SG	Admin Panel task
RDS SG	Customer + Admin + Lambda SG	PostgreSQL (5432)
ElastiCache SG	Customer Fargate SG	Redis (6379)
Bastion SG	Whitelisted admin IPs	SSH (22) entry
Lambda SG	N/A (outbound only)	RDS + SES access

This layered model ensures that even if one security group is misconfigured, adjacent layers prevent unauthorized access. S3 is not publicly accessible (OAC only).

2.5 VPC and Subnet Layout

Production VPC (10.0.0.0/16) is divided into three tiers across two Availability Zones:

Subnet	CIDR	AZ	Purpose
Public A	10.0.1.0/24	AZ-A	Public ALB, Bastion, NAT GW
Public B	10.0.2.0/24	AZ-B	Public ALB (multi-AZ)
Private App A	10.0.10.0/24	AZ-A	Fargate: Catalog, Cart, Checkout
Private App B	10.0.20.0/24	AZ-B	Fargate: Admin, Internal ALB
Private Data A	10.0.30.0/24	AZ-A	RDS primary, ElastiCache Redis
Private Data B	10.0.40.0/24	AZ-B	RDS standby replica

The data tier has no route to the internet - access is restricted to specific ports via SGs. Dev VPC (10.1.0.0/16) mirrors the same three-tier structure in a single AZ with reduced resources. No VPC peering exists between environments - complete network isolation.

3. Service-to-Infrastructure Mapping

This section maps each of the eight platform services defined in the project specification to its AWS infrastructure. For every service, we provide: the exact project requirement it addresses, the AWS components selected, a justification for why this combination was chosen, a list of alternatives that were considered and rejected (with reasons), and the cost impact on the monthly bill. This mapping ensures full traceability from requirement to infrastructure.

3.1 Product Catalog

Project Requirement

"Public-facing storefront serving product listings, images, categories, and search results to anonymous and authenticated customers."

AWS Components

Component	Configuration	Role
ECS Fargate	0.5 vCPU / 1 GB, Auto Scaling	Runs catalog API container
Public ALB	Multi-AZ, path: /api/products/*	Routes customer traffic
CloudFront + S3	OAC, edge caching	Serves product images & static assets
RDS PostgreSQL	Read queries via primary	Product data, categories, search

Why This Choice

Highest-traffic read service - every page view starts here. Fargate scales the catalog independently from other services. Static images are offloaded to S3 + CloudFront for edge caching. PostgreSQL's native full-text search (ts_vector, ts_query) powers product search without ElasticSearch, and JSONB handles variable product attributes (size, color, specs) without separate attribute tables.

Alternatives Rejected

- **EC2 + Auto Scaling Group:** Slower scaling (2-5 min instance launch vs seconds for Fargate tasks), requires AMI management, OS patching, and capacity planning. Over-provisioning wastes money; under-provisioning drops requests.
- **EKS (Kubernetes):** $73/month control plane cost per cluster ($146 for prod + dev). Provides powerful orchestration but adds operational complexity (kubectl, Helm, RBAC) disproportionate to the workload. Fargate achieves the same per-service scaling without cluster management overhead.
- **Lambda (API Gateway + Lambda):** Cold starts (100-500ms) create noticeable latency on product page loads. Not suitable for a consistently-accessed storefront. Better

suited for event-driven workloads like invoice generation.

Cost Impact: Fargate: ~$18/month (0.5 vCPU x 730hrs x $0.04048 + 1GB x 730hrs x $0.004445). S3 + CloudFront: ~$0.12/month (free tier covers 1TB transfer + 10M requests).

3.2 Shopping Cart

Project Requirement

"Session-aware cart service that persists items across page loads."

AWS Components

Component	Configuration	Role
ECS Fargate	0.5 vCPU / 1 GB, Auto Scaling	Runs cart API container
ElastiCache Redis	cache.t3.micro, AZ-A	Session store (sub-ms reads)
Public ALB	Path: /api/cart/*	Routes cart requests

Why This Choice

Cart sessions need sub-millisecond reads and automatic TTL expiration for abandoned carts (24h). ElastiCache Redis in the data subnet provides managed HA, patching, and backup. Unlike in-container Redis, sessions persist across Fargate task restarts. The checkout service reads cart data from the same Redis endpoint, avoiding inter-service API calls.

Alternatives Rejected

- **In-container Redis (sidecar or separate task):** If the Redis container restarts, all active cart sessions are lost. Customers would need to re-add items - unacceptable for an e-commerce platform. No automatic backup or failover.
- **RDS-backed sessions (database table):** 10-50x slower than Redis for session reads (~5-10ms vs ~0.1ms). Every add-to-cart action would hit the database, adding unnecessary load to the RDS instance that handles orders and inventory.
- **Client-side storage (cookies or localStorage):** Limited by cookie size (4KB), not shareable across devices, and vulnerable to tampering. Server-side sessions in Redis are more secure and support larger cart payloads.

Cost Impact: Fargate: ~$18/month. ElastiCache Redis (cache.t3.micro): ~$12.41/month.

3.3 Checkout & Payment

Project Requirement

"Orchestrates the checkout flow: validates cart contents, confirms stock, and records the order."

AWS Components

Component	Configuration	Role
ECS Fargate	0.5 vCPU / 1 GB, Auto Scaling	Runs checkout API container
RDS PostgreSQL	Read/write via primary	Records orders, checks stock
ElastiCache Redis	Shared with cart service	Reads cart contents at checkout
SQS Queue	Standard queue	Publishes order event for invoice
Public ALB	Path: /api/checkout/*	Routes checkout requests

Why This Choice

Most critical transaction path. Flow: (1) read cart from Redis, (2) validate stock in RDS, (3) record order, (4) publish to SQS for async invoice (<10ms, non-blocking), (5) return 'Order Confirmed.' The customer never waits for invoice generation, directly satisfying the requirement. Reading from Redis avoids inter-service API calls.

Alternatives Rejected

- **Synchronous invoice generation:** Generating a PDF and sending an email adds 2-5 seconds to the checkout response. This directly violates the project requirement and degrades the customer experience.
- **SNS instead of SQS:** SNS is a push-based notification service - if the Lambda function is temporarily unavailable, the message is lost. SQS provides durable message storage with configurable retry and dead-letter queue support, ensuring no order event is ever lost.

Cost Impact: Fargate: ~$18/month. SQS: $0 (free tier covers 1M requests/month).

3.4 Authentication

Project Requirement

"Handles customer registration, login, and session management. A separate admin authentication flow controls access to the inventory management panel."

AWS Components

Component	Configuration	Role
Amazon Cognito	Customer User Pool	Registration, login, JWT for customers

Amazon Cognito	Admin User Pool	Separate credentials for admin staff
ALB Integration	JWT validation at ALB	Token verification before reaching services

Why This Choice

Cognito provides managed registration, login, JWT issuance, email verification, password reset, and brute-force protection with zero auth code. Two User Pools give complete credential separation (customer vs admin). The ALB natively validates JWT tokens before requests reach Fargate. Free tier covers 50,000 MAU - eliminating both the cost of an auth container (~$18/month) and the security risks of custom-built authentication (the #1 source of web app vulnerabilities).

Alternatives Rejected

- **Custom auth service (ECS Fargate container):** Requires implementing every auth feature from scratch: password hashing, JWT lifecycle, email verification, password reset, brute-force protection. Adds ~$18/month in Fargate costs for a service that Cognito provides for free with better security guarantees.
- **Auth0 / Okta (third-party managed auth):** Similar functionality to Cognito but adds an external dependency outside the AWS ecosystem. Pricing is per-user beyond free tier (typically 7,000-7,500 free MAU vs Cognito's 50,000). No native ALB integration - requires custom middleware.
- **Embedded auth per service:** Duplicates auth logic across catalog, cart, and checkout. Inconsistent security policies, harder to audit, and violates separation of concerns.

Cost Impact: Cognito: $0/month (free tier: 50,000 MAU). Eliminates the need for a dedicated auth Fargate task, saving ~$18/month compared to a custom solution.

3.5 Inventory Admin Panel

Project Requirement

"Internal web interface... Must never be reachable from the public internet - accessible only via a controlled internal network entry point."

AWS Components

Component	Configuration	Role
ECS Fargate	0.25 vCPU / 0.5 GB	Runs admin panel container
Internal ALB	No public IP, private subnets	Routes internal admin traffic
Bastion Host	t3.micro, public subnet	SSH entry point for admin access

Cognito Admin Pool	Separate user pool	Application-level admin authentication

Why This Choice

Strongest security requirement. Three independent isolation layers:

- **Network (Internal ALB):** No public IP, no public DNS. Resolves to private IP (10.0.x.x) unreachable from outside the VPC.
- **Access (Bastion SSH tunnel):** Only reachable via 'ssh -L 8080:<internal-alb>:80 bastion'. Bastion SG restricts SSH to whitelisted admin IPs.
- **Auth (Cognito Admin Pool):** Even after SSH tunnel access, admin must authenticate against a separate Cognito User Pool.

Alternatives Rejected

- **AWS Client VPN + Internal ALB:** VPN costs ~$73/month base + $0.05/hour per active connection. Combined with the Internal ALB (~$18/month), this adds ~$91/month for admin access to a single internal tool. Disproportionate for a startup where 1-2 staff access the admin panel during business hours.
- **Public ALB with IP whitelist:** Technically accessible from the internet - violates 'must never be reachable from the public internet.' IP whitelists can be spoofed or become stale. Does not satisfy the requirement.
- **Direct Fargate access (no ALB):** Fargate tasks receive dynamic IPs on each deployment. Without a load balancer, there is no stable endpoint for the SSH tunnel to target.

Cost Impact: Fargate (0.25 vCPU): ~$9/month. Internal ALB: ~$18/month. Bastion (t3.micro): ~$7.59/month. Total admin infrastructure: ~$34.59/month.

3.6 Orders & Inventory Database

Project Requirement

"Relational database... Requires high availability with automatic failover, geographic redundancy, and regular automated snapshots."

AWS Components

Component	Configuration	Role
RDS PostgreSQL	db.t3.micro, Multi-AZ, 20GB gp3	Primary database (AZ-A)
RDS Standby	Synchronous replica (AZ-B)	Automatic failover target
RDS Cross-Region	Read replica in eu-west-1	Geographic redundancy (Ireland)
Automated Snapshots	35-day retention	Point-in-time recovery

Why This Choice

Multi-AZ satisfies all three requirements: synchronous standby in AZ-B (RPO = 0, failover 60-120s), automated snapshots with 35-day retention, and cross-region replica in eu-west-1 (Ireland) for geographic redundancy close to EU customers.

PostgreSQL over MySQL for e-commerce: (1) JSONB for flexible product attributes without separate tables, (2) built-in full-text search without ElasticSearch, (3) superior analytics for sales reports. The eu-west-1 replica provides both DR and low-latency EU reads - an Oregon replica would only provide DR within North America. Database is in private data subnets with SG access restricted to Fargate and Lambda SGs only.

Alternatives Rejected

- **Amazon Aurora PostgreSQL:** ~30% more expensive than standard RDS for equivalent instance sizes. No free-tier eligible instance class. For a startup seeking 'minimum investment to go live,' standard RDS satisfies every requirement at lower cost. Aurora is the natural upgrade path if the platform outgrows db.t3.micro.
- **Self-managed PostgreSQL on EC2:** No automatic failover - if the instance dies, the database is down until manual intervention. Must manually script backups, replication, patching, and monitoring. Directly contradicts the HA requirement and adds significant operational burden.
- **DynamoDB (NoSQL):** The project explicitly requires a 'relational database' for orders, inventory, and customer records. DynamoDB's key-value model would require significant workarounds for relational queries like 'show all orders for customer X with product details.' Foreign keys and ACID transactions are natural in PostgreSQL but complex in DynamoDB.

Cost Impact: RDS Multi-AZ (db.t3.micro): ~$27.88/month. Cross-region replica: ~$14.74/month. Total database: ~$42.62/month. Dev: ~$14.74/month (Single-AZ, no replica).

3.7 Invoice Generation

Project Requirement

"When an order is confirmed, a PDF invoice is automatically generated and emailed to the customer. This job must not block or delay the checkout response."

AWS Components

Component	Configuration	Role
SQS Queue	Standard, 4x retry	Buffers order events from checkout
SQS DLQ	Dead Letter Queue	Captures permanently failed messages

Lambda	Python, 128MB, 30s timeout	Generates PDF invoice on trigger
S3 Bucket	Invoice storage, lifecycle policy	Stores PDFs for re-download
SES	Verified domain	Emails invoice PDF to customer

Why This Choice

Checkout publishes to SQS and returns immediately (<200ms). Lambda generates the PDF, uploads to S3 (enabling re-download for returns/warranties), and emails a link via SES. SQS retries on Lambda failure; after max retries, messages move to a DLQ for inspection. No invoice is silently lost. Lambda scales from 0 to handle bursts. Entire pipeline is within AWS free tier at startup.

Alternatives Rejected

- **Background Fargate task (queue poller):** Runs 24/7 at ~$10-15/month even with zero invoices to generate. Must implement queue polling, retry logic, and dead-letter handling manually - all of which SQS + Lambda provides natively. At startup volume (~10 invoices/day), this container would be idle >99% of the time.
- **In-process async thread in checkout:** If the checkout container restarts or scales down mid-generation, the invoice is permanently lost with no retry mechanism. No durability guarantee. Violates the decoupling principle.
- **Step Functions orchestration:** Adds orchestration overhead for a simple linear pipeline (generate -> store -> email). Step Functions are better suited for complex workflows with branching logic and human approval steps. SQS + Lambda is simpler and cheaper for this use case.

Cost Impact: Effectively $0/month at startup volume. Lambda free tier: 1M requests + 400,000 GB-seconds. SQS free tier: 1M requests. SES: $0.10/1,000 emails. S3: ~$0.023/GB/month for stored PDFs.

3.8 Dev & Production Environments

Project Requirement

"The platform must run in two fully isolated environments... A misconfiguration or failure in dev must have zero impact on production."

AWS Components

Component	Configuration	Role
Production VPC	10.0.0.0/16, 2 AZs, 3-tier subnets	Full HA production env
Development VPC	10.1.0.0/16, 1 AZ, 3-tier subnets	Reduced-cost dev env
SSM Parameter Store	/prod/* and /dev/* paths	Environment-scoped secrets

ECR	Per-service repositories	Container image storage

Why This Choice

Separate VPCs with no peering provide the strongest network isolation within a single account. No network path exists between environments. Dev uses reduced resources: single AZ, 0.25 vCPU Fargate tasks, Single-AZ RDS, no cross-region replica.

SSM Parameter Store scopes secrets by environment (/prod/db/password vs /dev/db/password) with IAM roles preventing cross-environment reads. ECR stores per-service images; both environments use the same Terraform modules with different variable files for consistency.

Alternatives Rejected

- **Separate AWS accounts (AWS Organizations):** The project specifies 'a clean AWS account' (singular), implying the grading team uses one account. Multi-account adds IAM cross-account roles, separate billing, and requires the grading team to configure AWS Organizations - unnecessary complexity.
- **Same VPC with subnet isolation:** Directly violates 'completely isolated at the network level.' Environments would share route tables, NAT Gateways, and Internet Gateways. A misconfigured route in dev could disrupt production traffic. Subnets are not a security boundary - they share the same VPC routing domain.
- **Kubernetes namespaces (same cluster):** Namespaces provide logical isolation but not network isolation. Pods in dev namespace can reach pods in prod namespace by default. Network Policies can restrict this, but a single misconfigured policy breaks isolation. Not equivalent to separate VPCs.

Cost Impact: Dev VPC mirrors prod at reduced scale. Key savings: no Multi-AZ RDS (~$13/month saved), no cross-region replica (~$15/month saved), no Internal ALB (~$18/month saved), no WAF (~$9/month saved), smaller Fargate tasks (~$45/month saved), single AZ. Total dev: ~$117/month vs prod ~$225/month.

4. Cost Estimation

All prices are for the us-east-1 (N. Virginia) region using on-demand Linux pricing at 730 hours/month. Costs were verified against official AWS pricing pages (April 2026) and are independently verifiable at calculator.aws. The goal is to estimate the minimum investment to go live, as specified by the project requirements.

4.1 Production Environment - $225.49/month

Resource	Configuration	$/month
Fargate - Customer Svc	3x (0.5 vCPU, 1 GB) 24/7	$54.06
Fargate - Admin Panel	1x (0.25 vCPU, 0.5 GB) 24/7	$9.01
EC2 Bastion Host	1x t3.micro ($0.0104/hr)	$7.59
RDS PostgreSQL Multi-AZ	db.t3.micro + 20GB gp3	$27.88
RDS Cross-Region Replica	db.t3.micro, eu-west-1	$15.33
ElastiCache Redis	cache.t3.micro	$12.41
NAT Gateway	1 gateway + ~50GB data	$35.10
Public ALB	1 ALB + ~1 LCU average	$22.27
Internal ALB	1 ALB + ~0.5 LCU average	$18.11
Public IPv4 addresses	~4 IPs ($0.005/hr each)	$14.60
WAF + Route 53	1 ACL + 3 rules + 1 zone	$9.00
CloudFront + S3	Free tier + ~5GB storage	$0.12
Cognito	Free tier (50K MAU)	$0.00
Lambda + SQS + SES + S3	All within free tier	$0.01
TOTAL		**$225.49**

Fargate compute ($63.07) is the largest line item, but it replaces both EKS control planes ($73/month) and EC2 worker nodes ($61/month) from a Kubernetes-based architecture - a net saving of $71/month. The NAT Gateway ($35.10) is the second-largest cost, charged per-hour plus per-GB of data processed. The Internal ALB ($18.11) is the cost of admin panel isolation - the only way to provide a stable, non-public endpoint for the Fargate admin service.

4.2 Development Environment - $117.04/month

Resource	Configuration	$/month

Fargate - Customer App	1x (0.25 vCPU, 0.5 GB)	$9.01
Fargate - Dev Admin	1x (0.25 vCPU, 0.5 GB)	$9.01
EC2 Bastion Host	1x t3.micro ($0.0104/hr)	$7.59
RDS PostgreSQL Single-AZ	db.t3.micro + 20GB gp3	$14.74
ElastiCache Redis	cache.t3.micro	$12.41
NAT Gateway	1 gateway + ~25GB data	$33.98
Dev ALB	1 ALB + ~0.5 LCU average	$19.35
Public IPv4 addresses	~3 IPs ($0.005/hr each)	$10.95
TOTAL		**$117.04**

The development environment saves $108/month compared to production. Key savings: no Multi-AZ RDS ($13 saved), no cross-region replica ($15 saved), no Internal ALB ($18 saved), no WAF/Route 53 ($9 saved), smaller Fargate tasks (0.25 vCPU vs 0.5 vCPU, $45 saved), and combined customer services into a single task. The NAT Gateway ($33.98) remains the largest dev cost - it is required for Fargate tasks to pull container images from ECR and make AWS API calls.

4.3 Combined Summary and Cost Drivers

Environment	Monthly	Annual
Production	$225.49	$2,705.88
Development	$117.04	$1,404.48
Combined	**$342.53**	**$4,110.36**

The three largest cost categories account for approximately 60% of total spending:

- Fargate compute: $63 (prod) + $18 (dev) = $81/month (24%). Per-second billing for actual vCPU and memory - no idle worker nodes or cluster management fees.
- NAT Gateways: $35 (prod) + $34 (dev) = $69/month (20%). Charged per-hour plus per-GB of data processed. Required for private subnet internet access.
- RDS PostgreSQL: $43 (prod Multi-AZ + replica) + $15 (dev Single-AZ) = $58/month (17%). Managed database with automatic failover and geographic redundancy.

Compared to an EKS-based architecture, Fargate eliminates $146/month in Kubernetes control plane fees and $91/month in EC2 worker nodes - a combined saving of $237/month ($2,844/year). The serverless invoice pipeline (Lambda + SQS + SES + S3) costs $0.01/month at startup volume. Cognito authentication is free for up to 50,000 monthly active users. CloudFront's free tier (1TB transfer, 10M requests) covers static asset delivery at near-zero cost.

5. Design Methodology - Key Decisions

This section documents the six most consequential design decisions, showing the options considered, evaluation criteria, and the reasoning behind each choice. Every decision was evaluated against four criteria: (1) project requirements, (2) startup cost-effectiveness, (3) course material alignment, and (4) Phase 2/3 implementation impact.

5.1 Compute: ECS Fargate + Lambda

Options Considered

- Option A: ECS Fargate (serverless containers) for synchronous services
- Option B: Amazon EKS (managed Kubernetes) for all services
- Option C: EC2 instances with Docker / Docker Swarm

Decision: ECS Fargate for 4 synchronous services + Lambda for async invoice

ECS Fargate provides per-service auto scaling, zero-downtime rolling deployments, and health check-based task replacement - all without managing a cluster. Each service (catalog, cart, checkout, admin) runs as an independent Fargate service with its own task definition, scaling policy, and security group. Fargate bills per-second for actual vCPU/memory consumed, with no minimum commitment.

EKS was rejected primarily for cost: $73/month per cluster ($146 for prod + dev) just for the control plane, before any compute. EKS also adds operational complexity (kubectl, Helm charts, RBAC policies, worker node AMI management) that is disproportionate for four web services. Fargate achieves the same per-service scaling without cluster management overhead.

EC2 with Docker was rejected because it requires manual capacity planning, OS patching, and instance management. EC2 Auto Scaling Groups take 2-5 minutes to launch new instances compared to Fargate's seconds. Docker Swarm lacks managed health checks and has been effectively deprecated in favor of Kubernetes and ECS.

Lambda handles invoice generation because it is fundamentally different from the synchronous services: event-driven, bursty (perhaps 10 invocations/day), and completely free under the Lambda free tier (1M requests/month). Running a 24/7 container for this workload would cost ~$10-15/month with >99% idle time.

5.2 Environment Isolation: Separate VPCs

Options Considered

- Option A: Separate VPCs in the same AWS account (no peering)
- Option B: Separate AWS accounts (AWS Organizations)

- Option C: Same VPC with namespace/subnet isolation

Decision: Separate VPCs, no peering, same AWS account

The project requires 'completely isolated at the network level' and 'zero impact' from dev failures on production. Separate VPCs provide true network isolation: different CIDRs, different route tables, different security groups, different NAT Gateways. With no VPC peering, there is literally no network path between the two environments.

Separate AWS accounts were rejected because the project specifies 'a clean AWS account' (singular), implying the grading team will use a single account. Multi-account also adds IAM cross-account role complexity and separate billing.

Same-VPC isolation with subnet separation was rejected because it directly violates 'completely isolated at the network level.' Environments would share route tables, NAT Gateways, and Internet Gateways - a misconfigured route in dev could disrupt prod traffic.

5.3 Region Strategy: Single Region + CDN + Cross-Region DR

Options Considered

- Option A: Single region (us-east-1) + CloudFront CDN
- Option B: Multi-region active-active (full infrastructure in 2 regions)
- Option C: Single region + CloudFront + cross-region database replica

Decision: Option C - us-east-1 with CloudFront and eu-west-1 replica

Multi-region active-active would double every component: 2x VPCs, 2x Fargate services, 2x ALBs, 2x NAT Gateways, 2x RDS instances. For a startup seeking 'minimum investment to go live,' this is unjustifiable - it would more than double the monthly cost. Cross-region database synchronization is also the hardest problem in distributed systems.

us-east-1 was chosen because it is the cheapest AWS region with the broadest service availability. Latency to US customers is ~10ms; to European customers ~100ms. CloudFront closes this gap for static content by serving from edge locations near both populations. The cross-region RDS replica in eu-west-1 (Ireland) provides both geographic redundancy AND a performance benefit for European customers. Unlike a replica in another US region (e.g., us-west-2), the Irish replica can serve EU read traffic at low latency and provides a DR target geographically close to half the customer base.

5.4 Database: RDS PostgreSQL Multi-AZ

Options Considered

- Option A: RDS PostgreSQL Multi-AZ (managed, automatic failover)

- Option B: RDS MySQL Multi-AZ (managed, simpler)
- Option C: Amazon Aurora PostgreSQL (managed, higher performance)
- Option D: Self-managed PostgreSQL on EC2 (full control)

Decision: RDS PostgreSQL Multi-AZ (db.t3.micro)

RDS PostgreSQL Multi-AZ directly satisfies every database requirement with a single managed service: automatic failover in 60-120 seconds, automated snapshots with 35-day retention, and cross-region read replicas for geographic redundancy. The db.t3.micro instance class provides 2 vCPUs and 1 GB RAM with burstable performance - sufficient for startup-scale transaction volume.

PostgreSQL was chosen over MySQL for three e-commerce-specific advantages: (1) native JSONB support handles variable product attributes (clothing: size/color; electronics: specs) without separate attribute tables - a common e-commerce data modeling challenge; (2) built-in full-text search (ts_vector/ts_query) powers product search without a separate ElasticSearch cluster; and (3) wire-compatible with Aurora PostgreSQL, providing a seamless upgrade path if the platform outgrows db.t3.micro.

Aurora was rejected because it costs ~30% more than standard RDS with no free-tier eligible instance class. Self-managed PostgreSQL on EC2 was rejected because it lacks automatic failover, requires manually scripted backups and replication, and directly contradicts the HA requirements.

5.5 Invoice Pipeline: SQS + Lambda + S3 + SES

Options Considered

- Option A: SQS + Lambda + S3 + SES + DLQ (serverless, event-driven)
- Option B: Background Fargate task polling a queue (always-on)
- Option C: In-process async thread within the checkout service

Decision: SQS + Lambda + S3 + SES with Dead Letter Queue

The project explicitly requires 'this job must not block or delay the checkout response.' This maps directly to an asynchronous, event-driven architecture. SQS provides durable message storage - messages persist until processed, surviving Lambda failures. Lambda scales per-message and costs $0 at startup volume. S3 stores generated PDFs for customer re-download (essential for returns, warranties, and expense reporting). SES handles email at $0.10 per 1,000 emails.

A background Fargate task would run 24/7 at ~$10-15/month even with zero invoices to generate. It would also require implementing queue polling logic, retry mechanisms, and dead-letter handling manually - all of which SQS + Lambda provides natively. An in-process async thread within checkout is fragile: if the checkout container restarts mid-generation, the invoice is permanently lost with no retry mechanism.

5.6 Admin Panel Access: Internal ALB + Bastion SSH Tunnel

Options Considered

- Option A: Internal ALB (no public IP) + Bastion SSH tunnel + Cognito Admin Pool
- Option B: Public ALB with IP whitelist
- Option C: AWS Client VPN + Internal ALB

Decision: Internal ALB + Bastion + Cognito Admin Pool

The project's strongest requirement: 'must never be reachable from the public internet.' The Internal ALB has no public IP address, no public DNS record, and exists only within the private app subnets. There is no internet-routable path to reach it. The bastion host in the public subnet serves as the 'controlled internal network entry point' the project requires.

Administrators access the panel by SSHing into the bastion and creating an SSH tunnel to the Internal ALB. They then open their browser to localhost, which tunnels through the encrypted SSH connection. This provides a real web interface experience - more practical for non-technical warehouse staff than alternatives requiring specialized tools.

Defense-in-depth is achieved through three independent layers: (1) the Internal ALB has no public exposure at the network level, (2) the bastion's security group restricts SSH to whitelisted administrator IPs, and (3) the Cognito Admin User Pool requires separate admin credentials at the application level. All three layers must be bypassed to access the panel.

A public ALB with IP whitelist was rejected because it is technically reachable from the internet, violating the requirement. AWS Client VPN was rejected on cost: ~$73/month base + $0.05/hour per connection. For 1-2 staff accessing the admin panel during business hours, the SSH tunnel approach provides identical security at a fraction of the cost (~$18/month for the Internal ALB + $7.59/month for the bastion, which is already needed).

6. Conclusion

The ShopCloud architecture balances production-grade reliability with startup-appropriate cost discipline. Every design decision is justified on both technical and economic grounds, with alternatives explicitly evaluated and rejected where applicable.

The compute model uses ECS Fargate for synchronous services and Lambda for event-driven work, demonstrating workload-appropriate infrastructure selection. Fargate provides per-service auto scaling without cluster management overhead, while the serverless invoice pipeline (SQS + Lambda + S3 + SES + DLQ) operates at zero marginal cost. RDS PostgreSQL Multi-AZ delivers the required high availability, automatic failover, and geographic redundancy through a cross-region replica in eu-west-1 (Ireland), strategically placed to serve European customers.

Security is enforced through defense-in-depth at every layer: AWS WAF and Shield Standard filter malicious traffic at the CloudFront edge, Amazon Cognito validates JWT tokens at the ALB before requests reach services, eight security groups with SG-to-SG references enforce least-privilege access, three-tier subnet isolation separates public, application, and data resources, and the admin panel is triply isolated behind an Internal ALB, a bastion SSH tunnel, and a Cognito Admin User Pool. Environment isolation via separate VPCs (10.0.0.0/16 and 10.1.0.0/16) with no peering ensures development failures cannot propagate to production.

The estimated minimum investment to go live is $342.53 per month ($4,110 annually) for both environments - 25% less than an equivalent EKS-based architecture, primarily by eliminating $146/month in Kubernetes control plane fees and $91/month in EC2 worker nodes. The architecture is designed to be fully provisionable via Terraform from a single AWS account (Phase 2), monitorable via AWS CloudWatch (Phase 3), and evolvable as the platform grows. Clear upgrade paths exist: standard RDS to Aurora PostgreSQL for higher throughput, single-region to multi-region for lower global latency, and bastion SSH tunnel to AWS Client VPN for improved admin UX.

The architecture prioritizes the 'minimum investment to go live' while meeting every stated requirement without compromise. No service is over-engineered for current scale, and no requirement is left unsatisfied.